

Lesson 7 — Sparse Matrix Using C++ Classes

1. Where We Are in the Course

Before Lesson 7, you already learned how sparse matrices work in C-style programming.

What you already know

Lesson	Main idea
Lesson 1	A sparse matrix stores mostly zero values, so we store only non-zero values.
Lesson 2	Coordinate List representation stores each non-zero value as row, column, and value.
Lesson 3	A sparse matrix can be displayed as a full matrix using nested loops and a tracker variable.
Lesson 4	Sparse matrix addition uses a merge-style algorithm with <code>i</code> , <code>j</code> , and <code>k</code> .
Lesson 5	The <code>add()</code> function creates a result matrix and returns it as a pointer.
Lesson 6	The full C program uses <code>malloc</code> , <code>free</code> , and pointer-based functions.
Lesson 7	The sparse matrix program is converted into a C++ class.

Lesson 7 is the beginning of object-oriented sparse matrices.

The sparse matrix idea does not change.

We still store only non-zero values.

The main change is how we organize the program.

2. Main Idea of Lesson 7

In earlier lessons, we used separate structures and functions.

Example C-style use:

```
struct Sparse s;  
create(&s);  
display(s);
```

Here:

- `s` stores the data.
- `create()` is a separate function.
- `display()` is another separate function.

This style is called procedural programming.

In Lesson 7, we use a C++ class.

Example C++ class-style use:

```
SparseMatrix s1(5, 5, 5);  
s1.read();  
s1.display();
```

Here:

- `s1` is an object.
- The object stores the matrix data.
- The object also has functions that work on that data.

So the data and functions are kept together.

This is the object-oriented programming idea.

3. Procedural Style vs Object-Oriented Style

Procedural style

In procedural style, data and functions are separate.

```
struct Sparse s;  
create(&s);  
display(s);
```

The matrix is passed to functions.

Object-oriented style

In object-oriented style, data and functions are placed inside a class.

```
SparseMatrix s1(5, 5, 5);  
s1.read();  
s1.display();
```

The object calls its own functions.

Simple comparison

Procedural style	Object-oriented style
Uses <code>struct Sparse</code>	Uses <code>class SparseMatrix</code>
Functions are outside the structure	Functions are inside the class
Call <code>create(&s)</code>	Call <code>s1.read()</code>
Call <code>display(s)</code>	Call <code>s1.display()</code>
Data is handled from outside	Object manages its own data

4. Beginner Mental Model

Think of the class as a box.

Inside the box, we keep:

```
SparseMatrix  
├─ data: m, n, num, ele  
├─ function: read()  
└─ function: display()
```

The object knows:

- how many rows it has,
- how many columns it has,
- how many non-zero values it stores,
- where the non-zero values are stored,
- how to read values,
- how to display the full matrix.

So the class becomes a complete unit.

5. Important Sparse Matrix Reminder

A sparse matrix is a matrix where most values are zero.

Example:

```
1 0 0 0 0
0 1 0 0 0
0 0 1 0 0
0 0 0 1 0
0 0 0 0 1
```

This is a 5×5 matrix.

It has:

```
5 × 5 = 25 positions
```

But only 5 values are non-zero.

So we do not store all 25 positions.

We store only the 5 non-zero values.

6. Coordinate List Representation Reminder

In Coordinate List representation, every non-zero value is stored using three values:

```
row, column, value
```

Example:

```
0 0 1
1 1 1
2 2 1
3 3 1
4 4 1
```

This represents the full matrix:

```
1 0 0 0 0
0 1 0 0 0
0 0 1 0 0
0 0 0 1 0
0 0 0 0 1
```

Each stored record tells us where a non-zero value belongs.

7. The `Element` Structure

One non-zero value is stored using an `Element` structure.

```
struct Element {
    int i; // row index
    int j; // column index
    int x; // value
};
```

Meaning of each field

Field	Meaning
<code>i</code>	row index
<code>j</code>	column index
<code>x</code>	value stored at that position

Example:

```
{2, 3, 7}
```

This means:

```
At row 2, column 3, the value is 7.
```

So one `Element` does not store the whole matrix.

It stores only one non-zero value.

8. The SparseMatrix Class

The sparse matrix is now represented using a class.

```
class SparseMatrix {
private:
    int m;
    int n;
    int num;
    Element *ele;

public:
    SparseMatrix(int m, int n, int num);
    ~SparseMatrix();

    void read();
    void display();
};
```

The class has two main parts:

1. `private` data
2. `public` functions

9. Private Data Members

The private section stores the actual data of the sparse matrix.

```
private:
    int m;
    int n;
    int num;
    Element *ele;
```

Meaning of each private member

Member	Meaning
<code>m</code>	number of rows
<code>n</code>	number of columns
<code>num</code>	number of non-zero elements

Member	Meaning
<code>ele</code>	pointer to a dynamic array of <code>Element</code> records

Example:

```
m = 5
n = 5
num = 5
```

This means:

- the matrix has 5 rows,
- the matrix has 5 columns,
- the matrix stores 5 non-zero values.

Important:

```
num does not mean m × n.
```

For a 5×5 matrix:

```
m × n = 25
```

But if only 5 values are non-zero:

```
num = 5
```

10. Why the Data Members Are Private

The data members are private because they should not be changed directly from outside the class.

This is not allowed:

```
s1.m = 5;
```

Why?

Because `m` is private.

Instead, the object should manage its own data using its own functions.

Example:

```
s1.read();  
s1.display();
```

This protects the data and keeps the program organized.

This idea is called encapsulation.

Simple meaning of encapsulation

Encapsulation means:

Keep data and related functions together inside one class.

It also means:

Do not allow outside code to directly damage or misuse the internal data.

11. Public Functions

The public section contains functions that can be called from outside the class.

```
public:  
    SparseMatrix(int m, int n, int num);  
    ~SparseMatrix();  
  
    void read();  
    void display();
```

These are public because the user of the class needs to create, read, and display the matrix.

Public members in Lesson 7

Public member	Purpose
Constructor	Initializes the object and allocates memory

Public member	Purpose
Destructor	Releases dynamic memory
<code>read()</code>	Reads non-zero elements
<code>display()</code>	Prints the full matrix

12. Constructor

A constructor is a special function that runs automatically when an object is created.

In Lesson 7, the constructor is:

```
SparseMatrix(int m, int n, int num) {
    this->m = m;
    this->n = n;
    this->num = num;
    ele = new Element[this->num];
}
```

When we write:

```
SparseMatrix s1(5, 5, 5);
```

The constructor is automatically called.

It receives:

```
m = 5
n = 5
num = 5
```

Then it stores those values in the object.

It also creates the dynamic array:

```
ele = new Element[this->num];
```

Since `this->num` is 5, this creates:

```
ele[0]
ele[1]
ele[2]
ele[3]
ele[4]
```

So the object now has enough space for 5 non-zero elements.

13. Why We Need a Parameterized Constructor

A parameterized constructor receives values when the object is created.

Lesson 7 uses:

```
SparseMatrix(int m, int n, int num)
```

This is useful because the object needs to know:

- how many rows there are,
- how many columns there are,
- how many non-zero elements there are.

The number of non-zero elements is especially important because we need it to allocate memory.

```
ele = new Element[this->num];
```

Without knowing `num`, the program would not know how large the dynamic array should be.

14. The `this` Pointer

Inside the constructor, the parameter names are the same as the class data member names.

Example:

```
SparseMatrix(int m, int n, int num)
```

The class also has:

```
int m;  
int n;  
int num;
```

So there is a naming conflict.

To solve this, we use `this->`.

```
this->m = m;  
this->n = n;  
this->num = num;
```

Meaning

```
this->m = the object's m  
m       = the constructor parameter
```

So:

```
this->m = m;
```

means:

Store the parameter `m` inside the object's `m` variable.

Simple mental model

`this` means:

this current object

So `this->m` means:

the `m` that belongs to this object

15. Dynamic Memory Allocation Using `new`

In Lesson 6, the C version used `malloc`.

In Lesson 7, the C++ class version uses `new`.

```
ele = new Element[this->num];
```

This creates a dynamic array of `Element` records.

If:

```
num = 5
```

then the program creates space for:

```
5 Element records
```

Why use dynamic memory?

Because the number of non-zero elements may be different for each matrix.

One matrix may have 5 non-zero values.

Another matrix may have 20 non-zero values.

So we create memory based on `num`.

Important point

We do not allocate memory for `m × n` positions.

Wrong idea:

```
ele = new Element[m * n];
```

Correct idea:

```
ele = new Element[num];
```

Sparse matrices should store only non-zero elements.

16. Destructor

A destructor is a special function that runs automatically when an object is destroyed.

In Lesson 7, the destructor is:

```
~SparseMatrix() {  
    delete[] ele;  
}
```

The destructor releases the dynamic array created by the constructor.

The constructor creates memory:

```
ele = new Element[this->num];
```

The destructor releases memory:

```
delete[] ele;
```

This prevents memory leaks.

17. Why We Use `delete[]`

Because `ele` was created as an array:

```
new Element[this->num]
```

we must delete it as an array:

```
delete[] ele;
```

Wrong:

```
delete ele;
```

Correct:

```
delete[] ele;
```

Memory rule

Allocation	Deallocation
<code>new</code>	<code>delete</code>
<code>new[]</code>	<code>delete[]</code>
<code>malloc</code>	<code>free</code>

In this lesson:

```
new[] goes with delete[]
```

18. `read()` Function

The `read()` function reads the non-zero elements from the user.

```
void read() {  
    cout << "Enter non-zero elements as row column value:\n";  
  
    for (int i = 0; i < num; i++) {  
        cin >> ele[i].i >> ele[i].j >> ele[i].x;  
    }  
}
```

The function reads only non-zero elements.

It does not read every matrix position.

If `num = 5`, the loop runs 5 times.

It fills:

```
ele[0]
ele[1]
ele[2]
ele[3]
ele[4]
```

Each element receives:

```
row, column, value
```

Example input:

```
0 0 1
1 1 1
2 2 1
3 3 1
4 4 1
```

Stored result:

```
ele[0] = {0, 0, 1}
ele[1] = {1, 1, 1}
ele[2] = {2, 2, 1}
ele[3] = {3, 3, 1}
ele[4] = {4, 4, 1}
```

19. Why `read()` Does Not Use `this->`

Inside `read()`, there is no local variable named `num` that conflicts with the class member `num`.

So we can simply write:

```
for (int i = 0; i < num; i++)
```

and:

```
cin >> ele[i].i >> ele[i].j >> ele[i].x;
```

Using `this->` here would still work, but it is not necessary.

Example:

```
this->num  
this->ele
```

These are valid, but the lesson keeps the code simple.

20. `display()` Function

The `display()` function prints the sparse matrix as a full matrix.

Even though the object stores only non-zero elements, `display` prints all visible positions.

```
void display() {  
    int k = 0;  
  
    for (int i = 0; i < m; i++) {  
        for (int j = 0; j < n; j++) {  
            if (k < num && i == ele[k].i && j == ele[k].j) {  
                cout << ele[k].x << " ";  
                k++;  
            } else {  
                cout << "0 ";  
            }  
        }  
        cout << endl;  
    }  
}
```

21. How `display()` Works

The `display` function uses nested loops.

The outer loop moves through rows:

```
for (int i = 0; i < m; i++)
```

The inner loop moves through columns:


```
for (int j = 0; j < n; j++)
```

So the function visits every position:

```
(0,0), (0,1), (0,2), ...
```

For each position, it checks whether the current position matches the next stored non-zero element.

```
if (k < num && i == ele[k].i && j == ele[k].j)
```

If the condition is true:

```
cout << ele[k].x << " ";  
k++;
```

If the condition is false:

```
cout << "0 ";
```

22. The Tracker Variable `k`

The variable `k` tracks the current non-zero element.

```
int k = 0;
```

At the beginning:

```
k = 0
```

So the current stored element is:

```
ele[0]
```

After printing `ele[0]`, the program moves to the next stored element:

```
k++;
```

Now:

```
k = 1
```

So the current stored element is:

```
ele[1]
```

Simple mental model for `k`

Think of `k` as a finger pointing to the next non-zero value.

The loops move through the full matrix.

The variable `k` points to the next stored value that may need to be printed.

23. Why We Check `k < num`

The display condition is:

```
if (k < num && i == ele[k].i && j == ele[k].j)
```

The first part is:

```
k < num
```

This checks whether `k` is still inside the valid range of the array.

Example:

If:

```
num = 5
```

valid indexes are:

```
0, 1, 2, 3, 4
```

When:

```
k = 5
```

then:

```
ele[5]
```

is outside the array.

That would be unsafe.

So `k < num` prevents the program from reading outside the array.

24. Row-Major Order Assumption

The `display()` function assumes the non-zero elements are stored in row-major order.

Row-major order means:

1. Sort by row first.
2. If rows are equal, sort by column.

Good order:

```
0 0 1
1 1 1
2 2 1
3 3 1
4 4 1
```

Another good order:

```
0 1 5
0 3 9
2 0 4
3 2 8
```

Bad random order:

```
4 4 1
0 0 1
2 2 1
1 1 1
3 3 1
```

Why is random order bad?

Because `k` only moves forward.

The display function does not search the full `ele` array again and again.

So if the input order is random, some non-zero values may not print correctly.

25. Complete Lesson 7 Program

```
#include <iostream>
using namespace std;

struct Element {
    int i; // row index
    int j; // column index
    int x; // value
};

class SparseMatrix {
private:
    int m; // number of rows
    int n; // number of columns
    int num; // number of non-zero elements
    Element *ele; // dynamic array of non-zero elements

public:
    SparseMatrix(int m, int n, int num) {
        this->m = m;
        this->n = n;
        this->num = num;

        ele = new Element[this->num];
    }

    ~SparseMatrix() {
```

```

        delete[] ele;
    }

    void read() {
        cout << "Enter non-zero elements as row column value:\n";

        for (int i = 0; i < num; i++) {
            cin >> ele[i].i >> ele[i].j >> ele[i].x;
        }
    }

    void display() {
        int k = 0;

        for (int i = 0; i < m; i++) {
            for (int j = 0; j < n; j++) {
                if (k < num && i == ele[k].i && j == ele[k].j) {
                    cout << ele[k].x << " ";
                    k++;
                } else {
                    cout << "0 ";
                }
            }
            cout << endl;
        }
    }
};

int main() {
    SparseMatrix s1(5, 5, 5);

    s1.read();

    cout << "\nSparse Matrix:\n";
    s1.display();

    return 0;
}

```

26. Explanation of the Complete Program

Header file

```
#include <iostream>
```

This is needed for:

```
cin  
cout
```

Namespace

```
using namespace std;
```

This allows us to write:

```
cout  
cin
```

instead of:

```
std::cout  
std::cin
```

Element structure

```
struct Element {  
    int i;  
    int j;  
    int x;  
};
```

This stores one non-zero matrix value.

SparseMatrix class

```
class SparseMatrix {
```

This starts the class definition.

Private members

```
private:  
    int m;
```

```
int n;  
int num;  
Element *ele;
```

These store the internal matrix data.

Constructor

```
SparseMatrix(int m, int n, int num)
```

This initializes the object.

Destructor

```
~SparseMatrix()
```

This releases the dynamic memory.

read()

```
void read()
```

This reads the non-zero elements.

display()

```
void display()
```

This displays the full matrix.

main()

```
int main() {  
    SparseMatrix s1(5, 5, 5);  
  
    s1.read();  
  
    cout << "\nSparse Matrix:\n";  
    s1.display();  
}
```

```
    return 0;  
}
```

This creates one sparse matrix object, reads its values, and displays it.

27. Dry Run Example

Create the object:

```
SparseMatrix s1(5, 5, 5);
```

This calls the constructor.

The constructor receives:

```
m = 5  
n = 5  
num = 5
```

So the object stores:

```
this->m = 5  
this->n = 5  
this->num = 5
```

Then it creates:

```
ele = new Element[5];
```

So the object has space for 5 non-zero elements.

28. Example Input

Input:

```
0 0 1  
1 1 1  
2 2 1
```



```
3 3 1
4 4 1
```

This means:

Row	Column	Value
0	0	1
1	1	1
2	2	1
3	3	1
4	4	1

Stored form:

```
ele[0] = {0, 0, 1}
ele[1] = {1, 1, 1}
ele[2] = {2, 2, 1}
ele[3] = {3, 3, 1}
ele[4] = {4, 4, 1}
```

29. Expected Output

The matrix displayed is:

```
1 0 0 0 0
0 1 0 0 0
0 0 1 0 0
0 0 0 1 0
0 0 0 0 1
```

This is a diagonal matrix.

Only positions where row and column are equal contain `1`.

All other positions are zero.

30. Dry Run of `display()`

Stored elements:

```
ele[0] = {0, 0, 1}  
ele[1] = {1, 1, 1}  
ele[2] = {2, 2, 1}  
ele[3] = {3, 3, 1}  
ele[4] = {4, 4, 1}
```

Initial value:

```
k = 0
```

Position `(0, 0)`

Current position:

```
i = 0  
j = 0
```

Current stored element:

```
ele[0] = {0, 0, 1}
```

Check:

```
i == ele[k].i → 0 == 0 → true  
j == ele[k].j → 0 == 0 → true
```

So print:

```
1
```

Then:

```
k = 1
```

Position (0, 1)

Current position:

```
i = 0  
j = 1
```

Current stored element:

```
ele[1] = {1, 1, 1}
```

Check:

```
i == ele[k].i → 0 == 1 → false
```

So print:

```
0
```

The same happens for:

```
(0,2), (0,3), (0,4)
```

So the first row is:

```
1 0 0 0 0
```

Position (1, 1)

Now:

```
i = 1  
j = 1  
k = 1
```

Current stored element:

```
ele[1] = {1, 1, 1}
```

The position matches.

So print:

```
1
```

Then:

```
k = 2
```

The same pattern continues until all rows are printed.

31. Complexity Analysis

Constructor complexity

The constructor stores a few values and creates the dynamic array.

In the lesson checkpoint, it is treated as:

```
O(1)
```

because it mainly sets up the object.

A deeper memory-level discussion may treat array allocation differently depending on the system, but for this lesson, remember:

```
Constructor: O(1)
```

`read()` complexity

The `read()` function loops through every non-zero element once.

If there are `num` non-zero elements, the loop runs `num` times.

So:

```
read() time complexity = O(num)
```

The object already has the array, so the reading process fills the existing storage.

`display()` complexity

The `display()` function visits every position in the full matrix.

If the matrix has:

```
m rows  
n columns
```

then the number of visible positions is:

```
m × n
```

So:

```
display() time complexity = O(m × n)
```

Extra space in `display()`

The display function only uses:

```
i, j, k
```

So extra space is:

```
O(1)
```

32. Complexity Table

Operation	Time Complexity	Extra/Result Space	Reason
Constructor	O(1) in this lesson	O(num) for the dynamic array	Sets fields and allocates the element array

Operation	Time Complexity	Extra/Result Space	Reason
<code>read()</code>	$O(\text{num})$	$O(1)$ extra	Reads each non-zero element once
<code>display()</code>	$O(m \times n)$	$O(1)$ extra	Visits every visible matrix position
Sparse storage	—	$O(\text{num})$	Stores only non-zero elements
Full matrix storage	—	$O(m \times n)$	Stores every position, including zeros

Important idea:

Sparse storage saves memory, but displaying the full matrix still takes full matrix time.

33. Difference Between Lesson 6 and Lesson 7

Lesson 6 used C-style programming.

Lesson 7 uses C++ class-style programming.

Concept	Lesson 6	Lesson 7
Language style	C-style	C++ class style
Matrix storage	<code>struct Sparse</code>	<code>class SparseMatrix</code>
Memory allocation	<code>malloc</code>	<code>new[]</code>
Memory release	<code>free</code>	<code>delete[]</code>
Function style	Separate functions	Member functions
Create/read	<code>create(&s)</code>	<code>s1.read()</code>
Display	<code>display(s)</code>	<code>s1.display()</code>
Data access	using <code>.</code> and <code>-></code>	mostly inside the class

The core sparse matrix idea is the same.

Only the program organization changes.

34. Important Code Symbols

Symbol	Meaning
<code>class</code>	Used to define a C++ class
<code>private</code>	Members accessible only inside the class
<code>public</code>	Members accessible from outside the class
<code>Element</code>	Structure for one non-zero value
<code>SparseMatrix</code>	Class for the whole sparse matrix
<code>m</code>	Number of rows
<code>n</code>	Number of columns
<code>num</code>	Number of non-zero elements
<code>ele</code>	Pointer to dynamic array of non-zero elements
<code>this-></code>	Refers to the current object's member
<code>new[]</code>	Creates a dynamic array in C++
<code>delete[]</code>	Releases a dynamic array in C++
Constructor	Runs automatically when object is created
Destructor	Runs automatically when object is destroyed
<code>read()</code>	Reads non-zero elements
<code>display()</code>	Prints full matrix
<code>k</code>	Tracks the current non-zero element while displaying

35. Common Beginner Mistakes

Mistake 1: Thinking `num` means total matrix size

Wrong:

```
num = m × n
```

Correct:

```
num = number of non-zero elements
```

Example:

```
5 × 5 matrix = 25 total positions  
5 non-zero values = num is 5
```

Mistake 2: Allocating `m × n` elements

Wrong:

```
ele = new Element[m * n];
```

Correct:

```
ele = new Element[num];
```

Why?

Because sparse matrix storage should store only non-zero elements.

Mistake 3: Forgetting the destructor

Wrong idea:

```
Create dynamic memory but never release it.
```

Correct:

```
~SparseMatrix() {  
    delete[] ele;  
}
```

Why?

Because memory created using `new[]` should be released using `delete[]`.

Mistake 4: Using `delete` instead of `delete[]`

Wrong:

```
delete ele;
```

Correct:

```
delete[] ele;
```

Why?

Because `ele` points to an array.

Mistake 5: Forgetting `this->` when names conflict

In the constructor:

```
SparseMatrix(int m, int n, int num)
```

The parameter names match the class member names.

So write:

```
this->m = m;  
this->n = n;  
this->num = num;
```

This clearly separates object members from parameters.

Mistake 6: Entering elements in random order

Wrong order:

```
4 4 1  
0 0 1  
2 2 1
```

```
1 1 1
3 3 1
```

Correct row-major order:

```
0 0 1
1 1 1
2 2 1
3 3 1
4 4 1
```

Why?

Because `display()` uses `k` and moves forward only.

Mistake 7: Expecting Lesson 7 to add matrices

Lesson 7 does not focus on addition.

Lesson 7 focuses on:

- class creation,
- private data,
- constructor,
- destructor,
- `read()`,
- `display()`.

Sparse matrix addition using operator overloading comes in Lesson 8.

36. Important Rules from Lesson 7

Rule 1

Use a class to group data and functions together.

```
class SparseMatrix {
    // data and functions
};
```

Rule 2

Keep matrix data private.

```
private:
    int m, n, num;
    Element *ele;
```

Rule 3

Use a constructor to initialize the object.

```
SparseMatrix(int m, int n, int num)
```

Rule 4

Use `this->` when constructor parameters have the same names as data members.

```
this->m = m;
```

Rule 5

Allocate only `num` elements.

```
ele = new Element[this->num];
```

Rule 6

Use a destructor to release dynamic memory.

```
~SparseMatrix() {
    delete[] ele;
}
```

Rule 7

Use `read()` to input only non-zero elements.

```
s1.read();
```

Rule 8

Use `display()` to print the full matrix.

```
s1.display();
```

Rule 9

Enter elements in row-major order.

Sort by row first, then by column.

Rule 10

Remember that display still takes $O(m \times n)$ time.

37. Practice Example

Suppose we create:

```
SparseMatrix s1(3, 4, 3);
```

This means:

```
m = 3  
n = 4  
num = 3
```

Input:

```
0 1 5  
1 3 8  
2 0 6
```

Stored form:

```
ele[0] = {0, 1, 5}  
ele[1] = {1, 3, 8}  
ele[2] = {2, 0, 6}
```

Expected display:

```
0 5 0 0
0 0 0 8
6 0 0 0
```

Explanation:

- `{0, 1, 5}` means row 0, column 1, value 5.
 - `{1, 3, 8}` means row 1, column 3, value 8.
 - `{2, 0, 6}` means row 2, column 0, value 6.
 - Every other position is zero.
-

38. Quick Self-Check Questions

Question 1

What is the main purpose of Lesson 7?

Answer

To convert the sparse matrix program into a C++ class using private data, a constructor, a destructor, `read()`, and `display()`.

Question 2

What does one `Element` store?

Answer

One non-zero value using row, column, and value.

Question 3

What does `num` mean?

Answer

`num` means the number of non-zero elements.

It does not mean `m × n`.

Question 4

Why do we use `new Element[this->num]` ?

Answer

Because we need dynamic memory for exactly `num` non-zero elements.

Question 5

Why do we use a destructor?

Answer

To release the dynamic memory created by `new[]` .

Question 6

Why do we use `delete[] ele` instead of `delete ele` ?

Answer

Because `ele` points to a dynamic array.

Dynamic arrays created with `new[]` must be released with `delete[]` .

Question 7

What does `this->m = m` mean?

Answer

It means store the constructor parameter `m` inside the current object's `m` data member.

Question 8

What does `k` do in `display()` ?

Answer

`k` tracks the current stored non-zero element in the `ele` array.

Question 9

Why should elements be entered in row-major order?

Answer

Because the display function uses `k` moving forward only. If elements are random, the output may be wrong.

Question 10

What is the time complexity of `display()` ?

Answer

$O(m \times n)$

because it visits every visible matrix position.

39. Final Summary

Lesson 7 changes the sparse matrix program from C-style procedural code into C++ class-based code.

The sparse matrix idea stays the same:

Store only non-zero elements.

Each non-zero element is still stored as:

row, column, value

The new idea is that the matrix data and functions are placed together inside a class.

The class contains private data:

```
int m;  
int n;
```

```
int num;  
Element *ele;
```

It also contains public functions:

```
SparseMatrix(int m, int n, int num);  
~SparseMatrix();  
void read();  
void display();
```

The constructor initializes the object and allocates memory.

The destructor releases memory.

The `read()` function reads non-zero elements.

The `display()` function prints the full matrix using nested loops and a tracker variable `k`.

The most important lesson is:

```
Lesson 7 does not change the sparse matrix logic.  
It changes the program design.
```

We move from separate functions to a class where the sparse matrix manages its own data and behavior.

This prepares us for Lesson 8, where we will use operator overloading so that sparse matrices can be used with syntax like:

```
cin >> s1;  
cout << s1;  
sum = s1 + s2;
```